CRUD approach and delete an item from the list while a user is shopping, that item disappears forever. And losing traceability limits your options. It is possible, for example, that after some time in production, the marketing team might get the idea to focus on the things that have been removed from the shopping cart as a sign that the buyer is interested in the product but not convinced. In response, the company would make special offers to those customers. With a state approach, where you replace in situ, that wouldn't be possible if you didn't add a specific field for those deletes, which adds complexity.

Another example is how to deal with bugs. For example, let's assume that an error occurred while calculating the price in the shopping cart. A discount should be applied to a certain product, but the program has a bug and applies the wrong discounts. If you have stored only the values but not the events in your database, you have to do a lot of work to correct each wrong value. You must find every value in the database and apply the correct discount. On the other hand, if you keep the events, such as products added, you can feed your program with those events once the bug is fixed, and that will be sufficient. Each cart will end up with the correct discount. An event sourcing system allows you to fix the code and replay the events.

Reading the history of events lets you look at the same events in retrospect from different perspectives. It gives you the opportunity to travel through time, not only to prove that the system is correct but also to replay it if it is not. However, sourcing events comes at a cost: namely, the cost of reading the data. The events are stored as a list of data structures, usually as tuples, with an ID associated with each event.